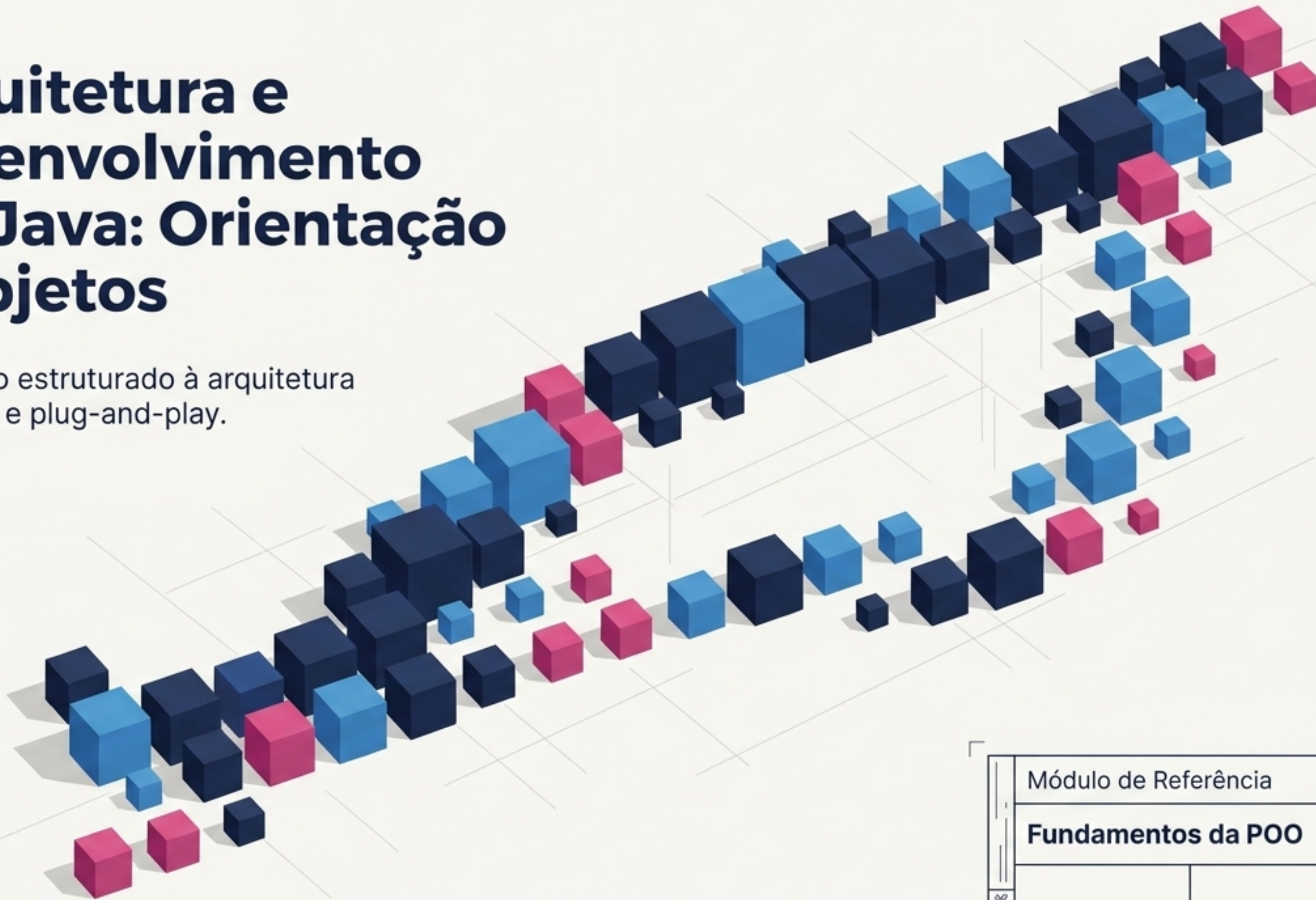


# Arquitetura e Desenvolvimento em Java: Orientação a Objetos

Do código estruturado à arquitetura escalável e plug-and-play.



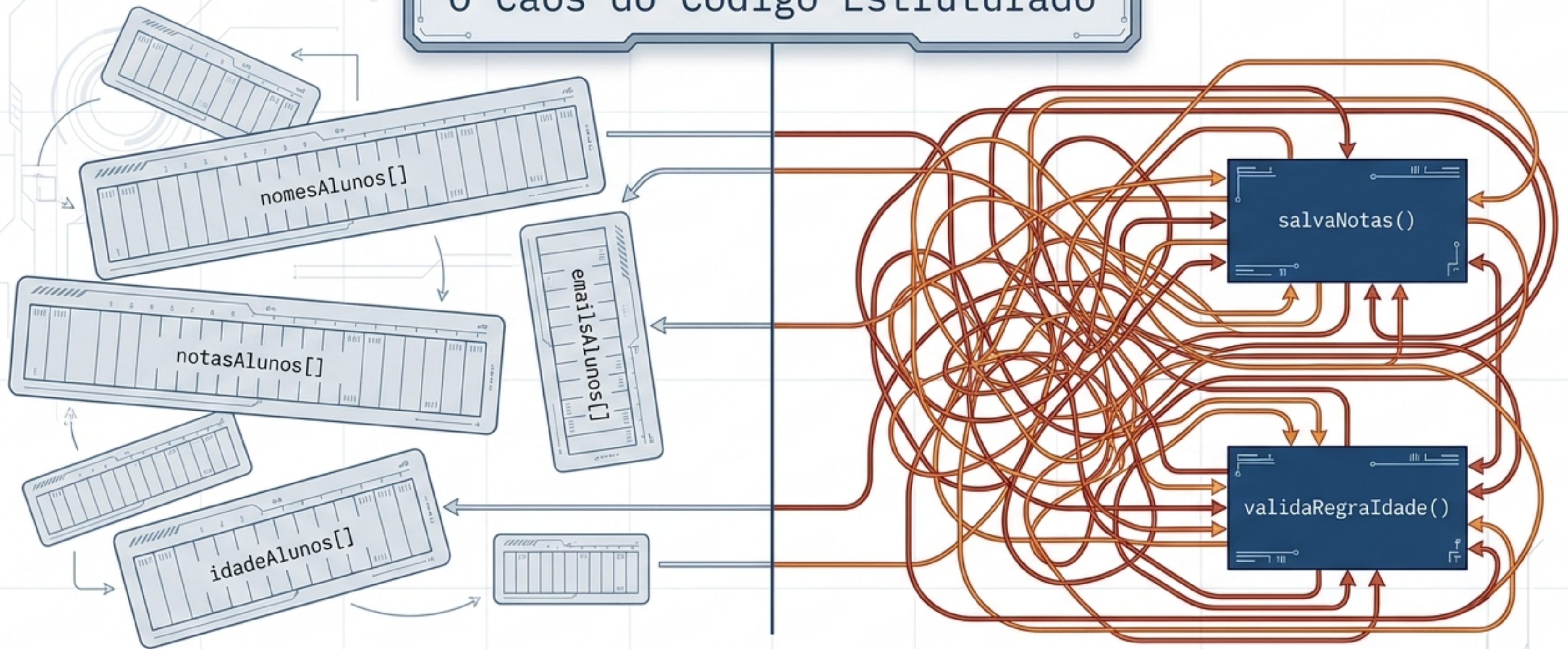
Módulo de Referência

**Fundamentos da POO**





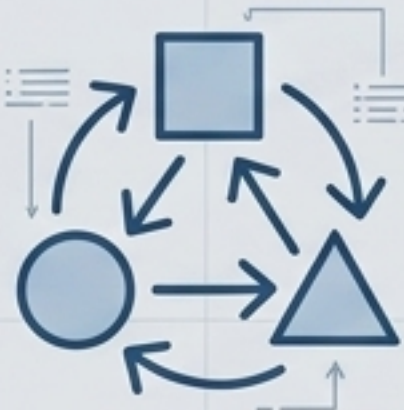
# 0 Caos do Código Estruturado



Quando a complexidade cresce, regras de negócio se espalham.  
Funções com alto acoplamento tornam o reaproveitamento de código quase impossível.  
Alterar um dado causa quebras em todo o sistema.



# Paradigmas: Estruturada vs. P00

|                     | Programação Estruturada                                                                                                  | Programação Orientada a Objetos (P00)                                                                                            |
|---------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Foco Principal      | Linearidade e fluxo de execução.<br> | Interação entre entidades independentes.<br> |
| Estrutura de Dados  | Dados (arrays) e Regras (funções) mantidos e processados separadamente.                                                  | Tudo é um objeto. Dados e métodos coexistem unificados na mesma entidade.                                                        |
| Manutenção e Escala | Alto acoplamento; alterações em dados podem quebrar múltiplas funções globais.                                           | Alto reuso e modularidade; facilita a evolução e protege a integridade do sistema.                                               |



# 0 Mundo Real Mapeado em Objetos



**Domínio: E-commerce**

1

**Entidades:**

Carrinho

Pedido

Produto

2

**Atributos (Estado):**

Data Criação, Status

3

**Operações (Ações):**

Qual o valor total?  
Calcular frete.



**Domínio: Sistema Escolar**

1

**Entidades:**

Aluno

Escola

Disciplina

2

**Atributos (Estado):**

Nome, E-mail, Notas

3

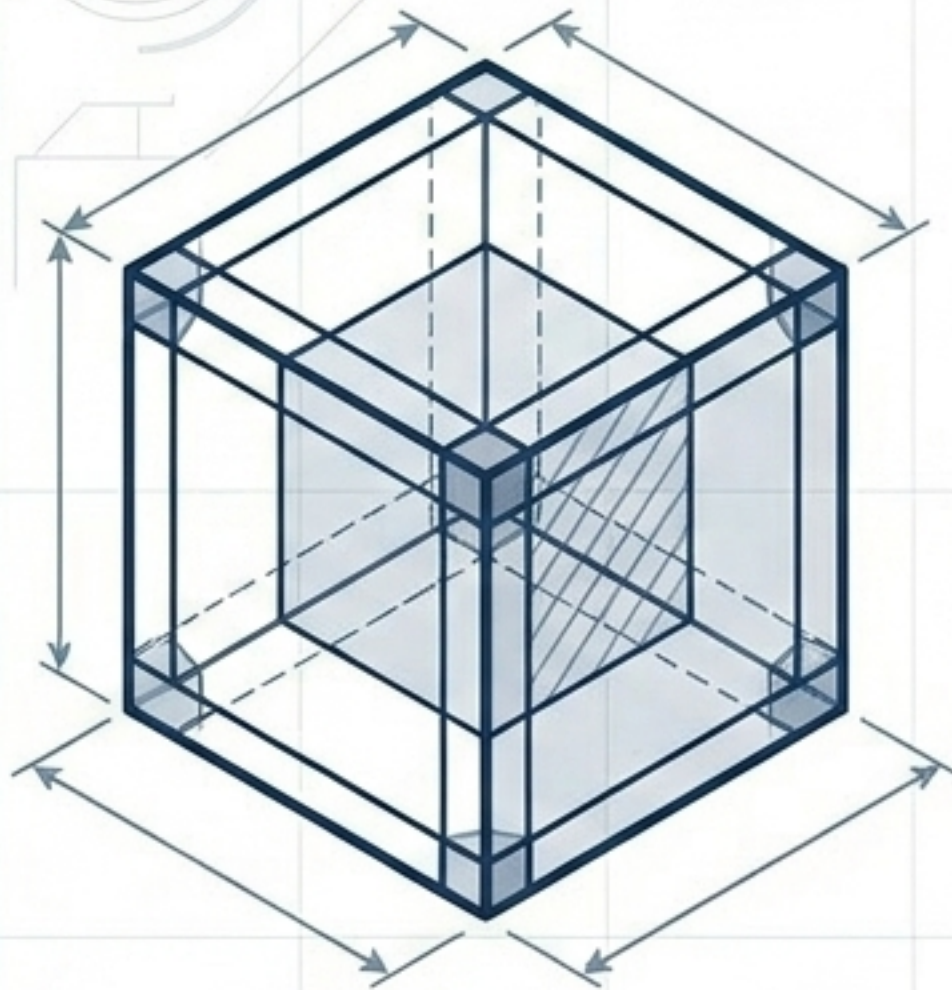
**Operações (Ações):**

Obter média das notas.  
Está aprovado?



# Da Classe para a Instância

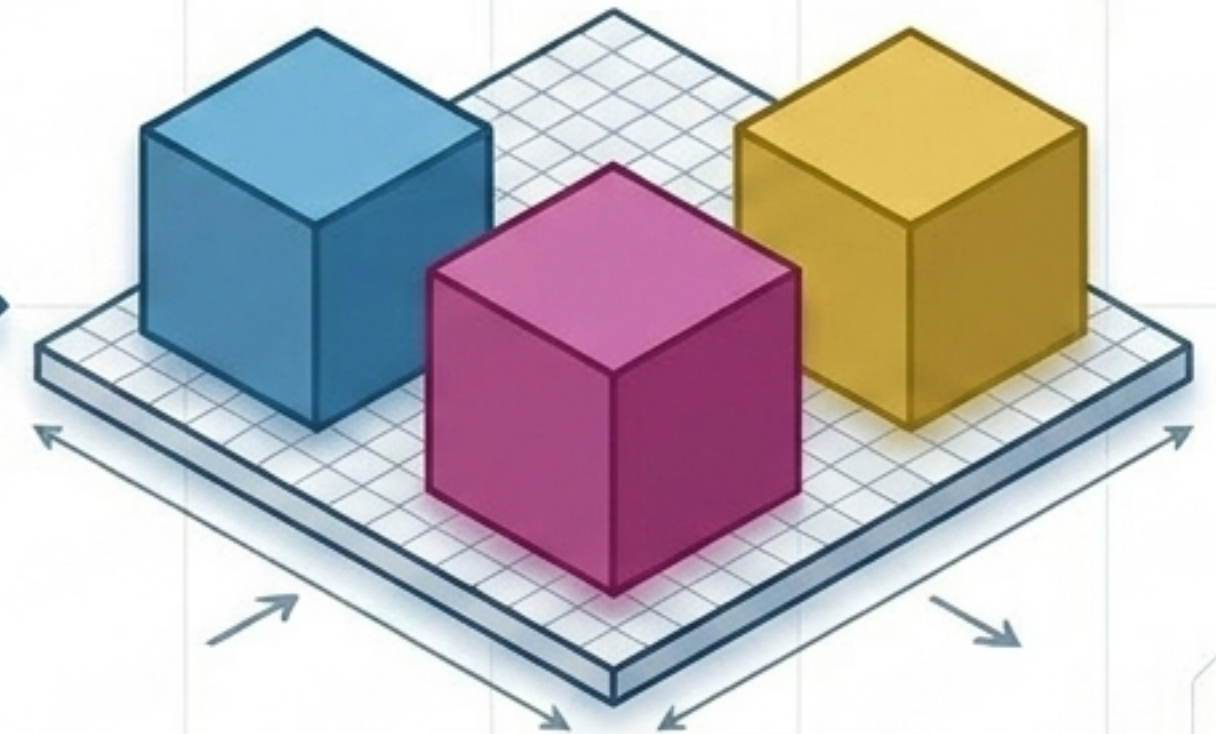
## A Classe (O Molde)



Código estático que define a estrutura (atributos e métodos).  
Não faz nada sozinha.



## Os Objetos (As Instâncias)



O que ganha vida no sistema.  
Entidades com identidades isoladas,  
operando no fluxo do programa.  
Ex: Aluno João, Aluno Maria.



# A Anatomia da Classe

**Atributos  
(Propriedades):**  
Variáveis que  
compõem o estado  
do objeto.

```
1 class Pedido {  
2     LocalDateTime dataHoraCriacao;  
3     Double desconto;  
4     StatusPedido status;  
5  
6     Double obterValorTotalComDesconto() {  
7         return valorTotal - desconto;  
8     }  
9 }
```

**Métodos  
(Comportamentos):**  
Operações que  
utilizam as  
propriedades do  
próprio objeto  
para executar  
regras de negócio.



# A Exceção à Regra: Métodos Estáticos



```
static Double somaValores(Double valorA,  
    Double valorB) {  
    return valorA + valorB;  
}
```

```
Double resultado = Pedido.somaValores(100, 200);
```

**Métodos estáticos (static)** pertencem à **Classe**, não ao **Objeto**. Recebem parâmetros externos e não precisam da instrução 'new' (instância) para serem executados. Ideais para operações utilitárias isoladas.



# Construtores: O Ponto de Partida

## 1 Instanciação



## 0 Comando

```
new Pedido()
```

Gatilho para iniciar a criação.

## 2 O Construtor

```
Pedido(Double desconto) {  
    this.desconto = desconto;  
}
```

Diferencia a propriedade da classe do parâmetro recebido.

## 3 Objeto Pronto



## Instância Populada

O objeto emerge pronto para uso seguro.

**Nota:** Se nenhum for definido, o Java provê um construtor 'Default' vazio.



# 0 Perigo do Acesso Direto



```
pedido.desconto = 50; // Bug grave gerado no sistema!
```

## 0 Risco da Quebra de Integridade

Se qualquer camada externa do sistema puder alterar atributos livremente (ex: mudar um desconto de 5 para 50), as regras de negócio falham. A integridade do objeto é destruída, gerando bugs em cadeia.



# 0 Cofre: Encapsulamento na Prática

```
public Double getDesconto()  
public void aplicarDesconto()
```

A única via de  
acesso autorizada

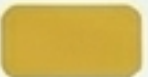
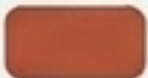
```
private Double desconto;
```

Protegido e  
inacessível  
diretamente

Encapsulamento é o princípio de blindagem. O mundo externo não toca nos atributos diretamente; toda comunicação deve ocorrer apenas através dos métodos controlados (Getters, Setters e Ações de Negócio).



## Matriz de Modificadores de Acesso no Java

| Modificador                                                                                                  | Classe | Pacote | Subclasse | Externo |
|--------------------------------------------------------------------------------------------------------------|--------|--------|-----------|---------|
| public                      | ✓      | ✓      | ✓         | ✓       |
| protected                   | ✓      | ✓      | ✓         | ✗       |
| default <br>(sem palavra) | ✓      | ✓      | ✗         | ✗       |
| private                   | ✓      | ✗      | ✗         | ✗       |

**A Regra de Ouro: Manter atributos 'private' e expor apenas os métodos 'public' estritamente necessários.**



# Interfaces: O Contrato Abstrato



```
1 public interface PedidoDatabaseGateway {  
2     void salvar(Pedido pedido);  
3     Pedido[] obterTodos();  
4 }
```

Uma Interface define **O QUE** deve ser feito (assinaturas de métodos), mas nunca **COMO** é feito (sem implementação). Elas atuam como contratos rigorosos, garantindo que diferentes camadas do sistema possam se comunicar sem depender do código interno uma da outra.



# Interfaces: Arquitetura Plug & Play

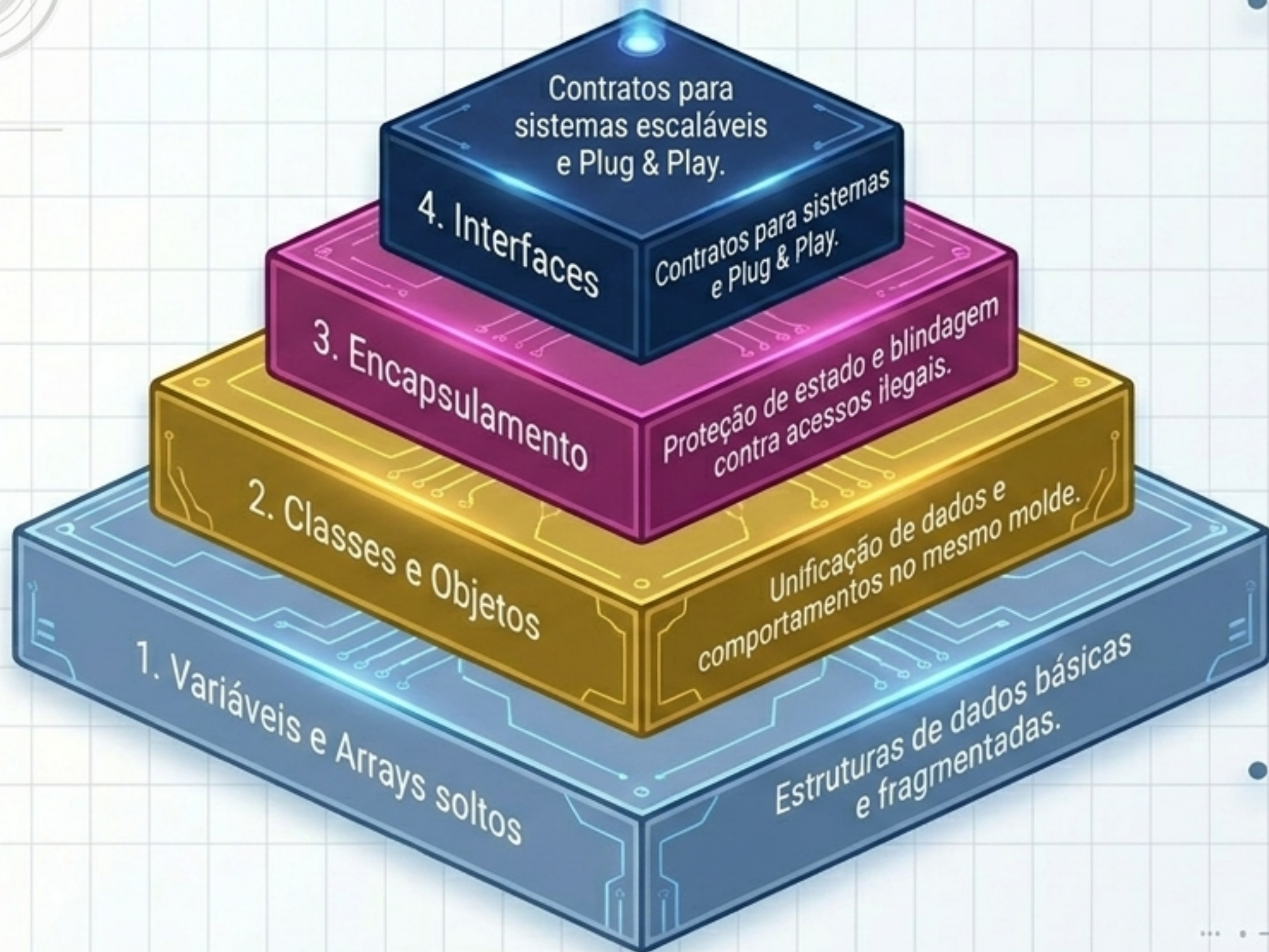


## Desacoplamento Perfeito

A regra de negócio superior não sabe qual banco de dados está sendo usado. Graças à interface, podemos trocar o cartucho do MySQL para o PostgreSQL instantaneamente, sem alterar sem alterar uma única linha do código principal.



# A Grande Imagem: Da Variável à Arquitetura



A Orientação a Objetos é um design pattern evolutivo para domar a complexidade e construir software robusto.



# Arquitetura Definida

- O paradigma estruturado linear e disperso é o passado.
- No design moderno, projete pensando que Tudo é um Objeto.
- Tranque e blinde seus domínios com forte Encapsulamento.
- Escale suas integrações externas usando Interfaces.

Preparado para o Hands-on.  
Hora de codificar.

